GESTION DE DONNÉES

Les concepts fondamentaux

Bases de données :

(notions de bases)

1. Définition d’une base de données

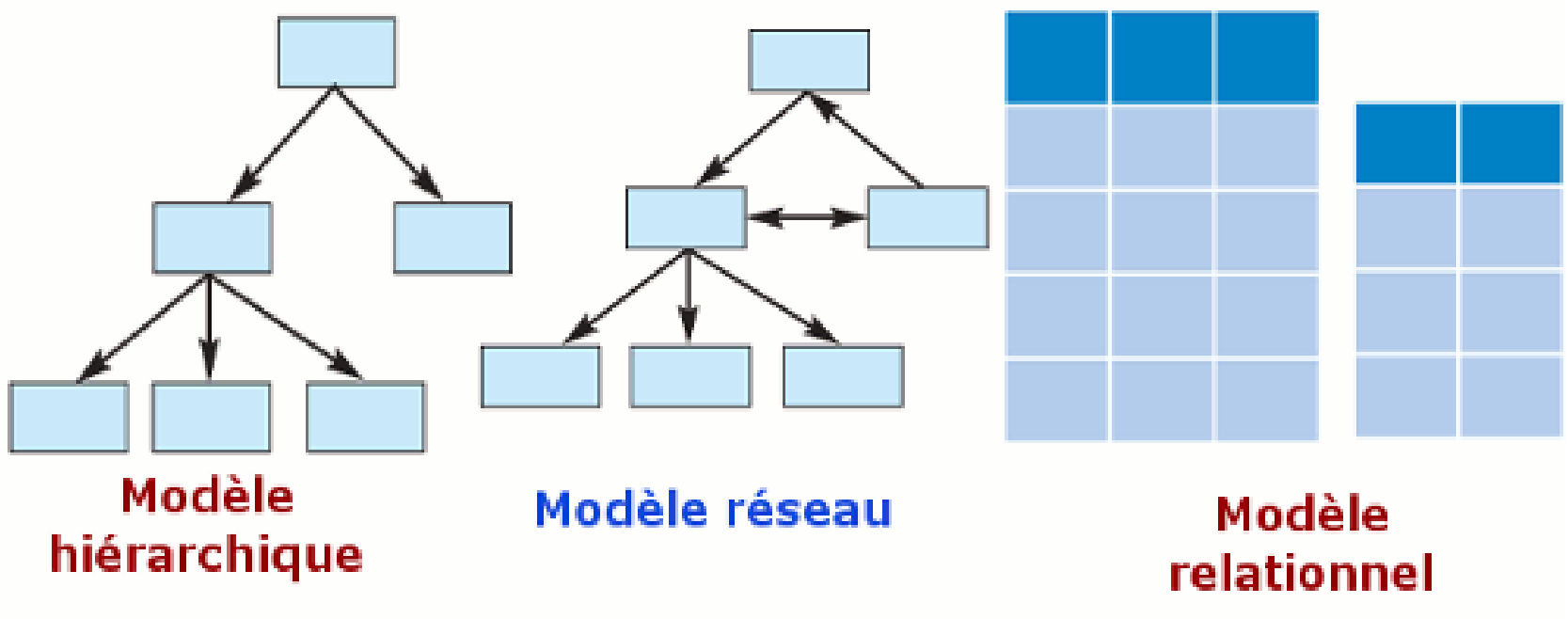
Une base de données est une collection de données structurées relatives à un ou plusieurs domaines du monde réel.

Elle est **persistante**, **structurée**, **non redondante** et **exhaustive**.

2. Intérêts de l'utilisation d'une base de données

- Centralisation
- Indépendance entre données et programmes
  surtout pour éviter les différances des lagunages de programmation
- Intégrité des données
  ensembles des règles qui permet la cohérence des données et qui (règles) doit être tjrs vérifiés
- Partage de données
- Intégration des liaisons entre données:
  les données sont reliées entre eux pour donner un sens

3. Les modèles de bases de données



3. Les modèles de bases de données

Modèle hiérarchique : organise les données dans une structure arborescente, où chaque enregistrement dispose d'un seul parent.

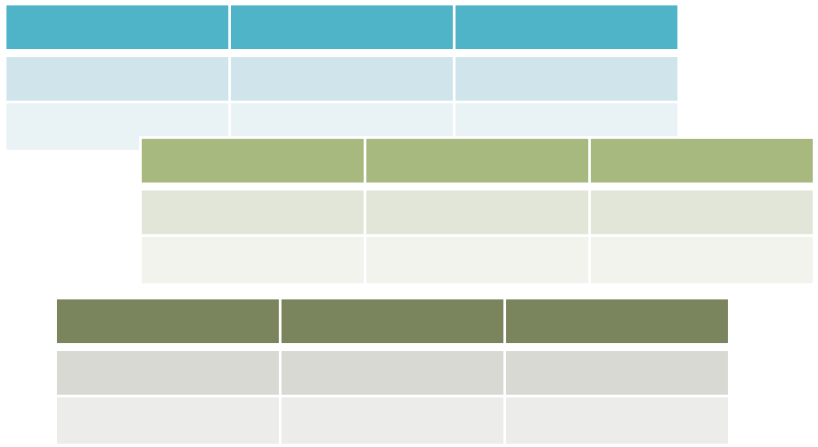
Modèle réseau : est une extension du modèle hiérarchique qui autorise des relations plusieurs-à-plusieurs entre des enregistrements liés.

Modèle relationnel : Consiste à présenter les objets et les liens à l'aide d'une structure appelée table.

Modèle orienté objet : Ce modèle définit une base de données comme une collection d'objets associés à des caractéristiques et des méthodes.

4. Modèle relationnel BDR

BDR



Contraintes d'integrités de :

- Table — Chaque table possède **PK**
- Domaine — Chaque colonne doit vérifier son **domaine De définition** (age > 18)
- Référentielle — (que dans le cas de plus qu'un table): **FK** (Table mère table Fils)

Notion de Système de Gestion de Bases de Données

(SGBD)

1. Définition d’un SGBD

Un Système de Gestion de Base de Données (SGBD) :

est un logiciel qui permet de **créer, organiser, modifier et consulter** des données de manière efficace et sécurisée.

En gros, c’est comme une **bibliothèque numérique** où les informations sont rangées dans des “étagères” (tables), et le SGBD aide à les retrouver facilement.



2. Les fonctionnalités d'un SGBD

- La définition des données
- La manipulation des données
- L'intégrité des données
- La gestion des accès concurrents
- La confidentialité

Structure d’une BDR

Base de Données Relationnelle

(SGBD)

1. Notion de table (table, entité, sujet, relation)

Les données d'une **BD** sont réparties sur un ensemble de tables.

Une table est un ensemble de données relatives à un même sujet (ou *entité*) et structurées sous forme de tableau.

Exemple : Structure de la table Personne

Colonne (champ)

CIN	Nom	Prenom	DateNaiss
05584123	Tounsi	Ahmed	12/01/2005
02774412	Karim	Thabet	23/05/1988
05893320	Selmi	Fahmi	08/07/2008

Lignes (enregistrement)

2. Notion de colonne (champ, attribut, propriété)

Dans une table, une colonne correspond à une propriété élémentaire de l'objet décrit par cette table.

Une colonne est décrite par :

- *un nom,*
- *un type de données,*
- *une taille,*
- *une valeur par défaut,*
- *un indicateur de présence obligatoire*
- *et une règle indiquant les valeurs autorisées.*

3. Notion de ligne (enregistrement, occurrence, n-uplet)

Une ligne correspond à une occurrence du sujet représenté par la table.

Exemple :

La table personne contient 3 lignes correspondant chacune à une personne.

4. Notion de clé primaire

La ***clé primaire*** d'une table est une colonne ou un groupe de colonnes permettant d'identifier de façon unique chaque ligne de la table.

La connaissance de la valeur de la clé primaire permet de connaître sans ambiguïté les valeurs des autres colonnes de la table.

5. Notion de clé étrangère

Les différents sujets de chaque domaine sont généralement interreliés par des liens (association).

Les liens entre les sujets doivent se retrouver dans la base de données.

6. Représentation d'une BD

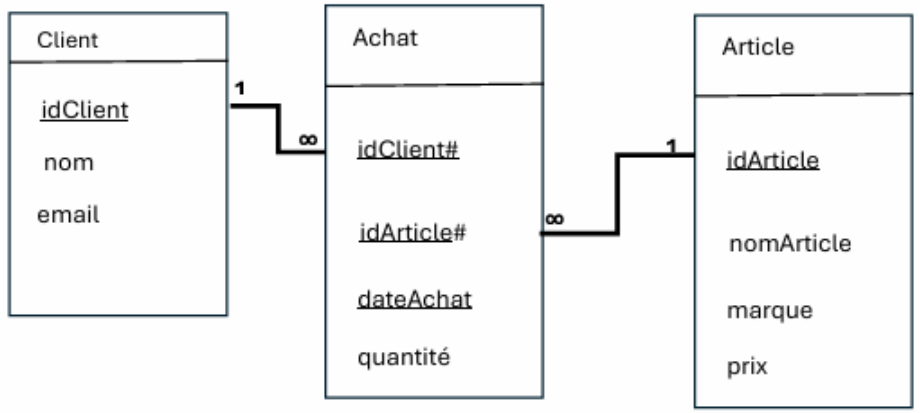
la représentation textuelle

client(**idClient**, nom, email)

article(**idArticle**, nomArticle, marque, prix)

achat (**idClient#, idArticle#,** dateAchat, quantité)

La représentation graphique



Remarque:

On Adopte, pour la représentation textuelle et graphique d'une BDR, la convention suivante:

- *clé primaire soulignée*
- *clé étrangère suivie du symbole #*

Manipulations sur la structure d'une base de données :

(Langages de **Définition des données**)

1. Langage SQL :

SQL (***Structured Query Language***) est un langage standard utilisé pour interagir avec les bases de données relationnelles.

Il permet d'effectuer diverses opérations comme la **création**, la **modification**, la **gestion** et la requête des données stockées dans des bases de données sous forme de tables.



2. Définition des bases :

La commande CREATE DATABASE est utilisée pour créer une nouvelle base de données dans un système de gestion de bases de données (SGBD) comme MySQL

```
CREATE DATABASE Bd_Letraditionnel;
```

La commande DROP DATABASE est utilisée pour supprimer une base de données entière, ainsi que toutes les tables et données qu'elle contient. Cette opération est irréversible et doit être utilisée avec précaution, car elle supprime définitivement toutes les données de la base de données.

```
DROP DATABASE Bd_Letraditionnel;
```

3. Définition des tables :

(Langages de **Définition des données**)

a. Création d'une table

La commande **CREATE TABLE** en SQL est utilisée pour créer une nouvelle table dans une base de données.

Elle permet de définir la structure de la table en spécifiant les colonnes, leurs types de données et d'autres contraintes comme les clés primaires, les clés étrangères, les valeurs par défaut, etc.

```
CREATE TABLE NomTable (
 Colonne1 TypeDeDonnée (taille) [Contraintes],
 Colonne2 TypeDeDonnée (taille) [Contraintes],
 , [contraintes définition complète]
);
```

b. Les types des données en SQL :

Type SQL	Description	Exemple d'utilisation	Exemple d'utilisation
INT	Entier sur 4 octets (32 bits), utilisé pour stocker des nombres entiers.	INT : Stocke des valeurs entières comme 100, -50, 2023.	*AGE INT;*
DECIMAL	Nombre décimal avec une précision fixe (nombre total de chiffres) et une échelle (chiffres après la virgule).	DECIMAL(10, 2) : Nombre avec jusqu'à 10 chiffres au total, dont 2 après la virgule.	PRIX DECIMAL(10, 2);
CHAR	Chaîne de caractères de longueur fixe. Si la longueur n'est pas atteinte, la chaîne est complétée par des espaces.	CHAR(10) : Chaîne fixe de 10 caractères.	CODE CHAR(10); pour des codes de produit de longueur fixe.
VARCHAR	Chaîne de caractères de longueur variable. La taille est flexible jusqu'à une limite définie.	VARCHAR(50) : Chaîne de longueur variable jusqu'à 50 caractères.	NOM VARCHAR(50); pour stocker des noms.
TEXT	Utilisé pour de longues chaînes de caractères, comme des paragraphes ou des descriptions.	TEXT : Texte long sans limite de taille prédéfinie.	DESCRIPTION TEXT; pour des descriptions détaillées.
DATE	Représente une date au format AAAA MM-JJ (année-mois-jour).	DATE : Stocke une date	DATE_NAISSANCE DATE; avec une valeur comme 1990-05-12.
TIME	Représente une heure au format HH:MM (heure:minute).	TIME : Stocke l'heure.	HEURE_ENTRÉE TIME; avec une valeur comme 14:30:00
DATETIME	Combine une date et une heure au format AAAA-MM-JJ HH:MM	DATETIME : Stocke une date et une heure	CREATION DATETIME; avec une valeur comme 2023-12-01 14:30:00.

c. Les contraintes :

Contrainte	Description	Exemple d'utilisation
PRIMARY KEY	Assure que chaque enregistrement dans une table est unique et non nul.	ClientID INT PRIMARY KEY
NOT NULL	Empêche une colonne de contenir des valeurs nulles (obligatoire).	Nom VARCHAR(50) NOT NULL
UNIQUE	Assure que toutes les valeurs d'une colonne (ou d'un ensemble de colonnes) sont uniques dans la table.	Email VARCHAR(100) UNIQUE
DEFAULT	Définit une valeur par défaut pour une colonne si aucune valeur n'est spécifiée lors de l'insertion.	DateInscription DATE DEFAULT "1990- 01-01"
REFERENCES	Déclare une clé étrangère (foreign key) qui fait référence à une colonne d'une autre table pour garantir l'intégrité référentielle.	ClientID INT REFERENCES Client(ClientID);

c. Contraintes définition complète: FOREIGN KEY

- La contrainte FOREIGN KEY (clé étrangère) est utilisée pour lier deux tables.
- Elle fait référence à une colonne ou à un ensemble de colonnes dans une autre table, généralement à une PRIMARY KEY .
- Elle garantit *l'intégrité référentielle* entre les tables.

Cela signifie que la valeur de la clé étrangère dans la table enfant doit exister dans la table parent.

c. Contraintes définition complète: FOREIGN KEY

```
CREATE TABLE NomTable (
 Colonne1 TypeDeDonnée (taille) [Contraintes],
 Colonne2 TypeDeDonnée (taille) [Contraintes],
 , FOREIGN KEY (ColonneFK) REFERENCES TableParent(ColonneParent) );
```

Exemple :

```
CREATE TABLE Commande (
 CommandeID INT PRIMARY KEY,
 ClientID INT,
 DateCommande DATE,
 FOREIGN KEY (ClientID) REFERENCES Client(ClientID)
);
```

Remarques :

Les clauses ***ON UPDATE CASCADE*** et ***ON DELETE CASCADE:***

sont utilisées en conjonction avec les contraintes de clé étrangère (FOREIGN KEY) dans SQL.

Elles définissent le comportement à adopter lorsqu'une ligne dans la table parent est mise à jour ou supprimée, en synchronisant automatiquement les modifications dans la table enfant.

Exemple :

```
CREATE TABLE Clients (
 ClientID INT PRIMARY KEY,
 Nom VARCHAR(100)
);

CREATE TABLE Commandes (
 CommandeID INT PRIMARY KEY,
 ClientID INT,
 FOREIGN KEY (ClientID) REFERENCES Clients(ClientID)
 ON UPDATE CASCADE  ON DELETE CASCADE
);
```

Explication :

- `ON UPDATE CASCADE` :

Si l'*ID* d'un client dans Clients est mis à jour,

la colonne *ClientID* dans Commandes sera automatiquement mise à jour.

- `ON DELETE CASCADE` :

Si un client est supprimé de Clients,

toutes ses commandes seront également supprimées de Commandes.

c. Contraintes définition complète: CHECK

```
CREATE TABLE NomTable (
 Colonne1 TypeDeDonnée CHECK (condition),
 Colonne2 TypeDeDonnée
);
```

Exemple :

```
CREATE TABLE Commande (
 ProduitID INT PRIMARY KEY,
  Nom VARCHAR(100) NOT NULL,
  Prix DECIMAL(10, 2),
  QuantitéStock INT CHECK (QuantitéStock >= 0)
  -- QuantitéStock doit être supérieure ou égale à 0
);
```

c. Contraintes définition complète: CHECK (Opérateurs de comparaison SQL)

Opérateur	Description	Exemple SQL avec CHECK	Explication de l'exemple
=	Vérifie l'égalité entre deux valeurs	CHECK (Age = 30)	Restreint l'âge à la valeur exacte de 30.
< > ou ! =	Vérifie l'inégalité (différent de)	CHECK (Age <> 30)	Interdit les enregistrements où l'âge est exactement 30.
>	Vérifie si la valeur de gauche est supérieure à celle de droite	CHECK (Age > 18)	Permet uniquement des âges supérieurs à 18.
<	Vérifie si la valeur de gauche est inférieure à celle de droite	CHECK (Age < 65)	Permet uniquement des âges inférieurs à 65.
> =	Vérifie si la valeur de gauche est supérieure ou égale à celle de droite	CHECK (Salaire >= 3000)	Garantit que le salaire est d'au moins 3000.
< =	Vérifie si la valeur de gauche est inférieure ou égale à celle de droite	CHECK (Salaire <= 10000)	Garantit que le salaire ne dépasse pas 10 000.
BETWEEN	Vérifie si une valeur est comprise entre deux valeurs	CHECK (Age BETWEEN 18 AND 65)	Restreint l'âge à être entre 18 et 65 inclus.
IN	Vérifie si une valeur est présente dans une liste de valeurs	CHECK (Statut IN ('Actif', 'Inactif'))	Limite le statut aux valeurs "Actif" ou "Inactif".

c. Contraintes définition complète: CHECK (pérateurs Logique SQL)

Opérateur	Description	Exemple	Explication de l'exemple
AND	Renvoie vrai si toutes les conditions sont vraies	CHECK (Age >= 18 AND Age <= 65)	Limite l'âge à être entre 18 et 65.
OR	Renvoie vrai si au moins une condition est vraie	CHECK (Sexe = 'F' OR Sexe = 'M')	Permet que le sexe soit soit "F" soit "M".
NOT	Renverse le résultat d'une condition	CHECK (NOT (Age < 18))	Interdit les enregistrements où l'âge est inférieur à 18.

c. Contraintes définition complète: CLÉ PRIMAIRE Composée

Une clé primaire composée est une clé primaire qui est définie sur plusieurs colonnes (ou champs) d'une table.

Cela signifie que la combinaison de ces colonnes doit être unique et non nulle pour chaque enregistrement, mais aucune de ces colonnes ne peut, à elle seule, identifier de manière unique une ligne dans la table.

Pourquoi utiliser une clé primaire composée ?

- Une clé primaire composée est utilisée lorsque plusieurs colonnes ensemble définissent de manière unique un enregistrement, mais qu'aucune colonne prise individuellement ne le fait.

- C'est souvent utilisé dans les tables de relation pour modéliser des associations entre plusieurs tables.

c. Contraintes définition complète: CLÉ PRIMAIRE Composée

Exemple:

Imaginons une table Inscription qui enregistre l'association entre des étudiants et des cours.
Chaque étudiant peut suivre plusieurs cours, et chaque cours peut être suivi par plusieurs étudiants.
Pour garantir qu'un étudiant ne s'inscrit pas plusieurs fois au même cours, nous allons utiliser une clé primaire composée des colonnes EtudiantID et CoursID.

```
CREATE TABLE Inscription (
 EtudiantID INT,
 CoursID INT,
 DateInscription DATE,
 PRIMARY KEY (EtudiantID, CoursID),
FOREIGN KEY (EtudiantID) REFERENCES Etudiant(EtudiantID),
FOREIGN KEY (CoursID) REFERENCES Cours(CoursID)
);
```

d. Supprimer une table :

La requête SQL DROP TABLE est utilisée pour supprimer une table existante dans une base de données, ainsi que toutes les données qu'elle contient. Une fois exécutée, cette opération est irréversible, c'est-à-dire que la table et ses données sont définitivement supprimées.

```
DROP TABLE NomTable;
```

Cette commande supprime la table Client ainsi que toutes les données et les contraintes associées à cette table.

Remarque :

Si la table a des relations (clés étrangères) avec d'autres tables, il peut être nécessaire de désactiver ou de supprimer ces relations avant d'exécuter la commande ***DROP TABLE***

4. Définition des colonnes :

La commande SQL **ALTER TABLE** permet de modifier la structure d'une table existante dans une base de données.

Cela inclut l'ajout, la suppression, la modification, ou le renommage de colonnes.

Opération	Syntaxe	Exemple
Ajouter une colonne	**ALTER TABLE NomTable** ADD COLUMN NomColonne TypeDeDonnée;	ALTER TABLE Client ADD COLUMN Email VARCHAR(100);
Supprimer une colonne	**ALTER TABLE NomTable** DROP COLUMN NomColonne;	ALTER TABLE Client DROP COLUMN Adresse;
Modifier le type d'une colonne (Version 01)	**ALTER TABLE NomTable** ALTER (MODIFY) COLUMN NomColonne NouveauType;	ALTER TABLE Client MODIFY COLUMN Age VARCHAR(3);
Modifier le type d'une colonne (Version 02)	**ALTER TABLE NomTable** MODIFY COLUMN NomColonne NouveauType;	
Renommer une colonne (Version 01)	**ALTERTABLE NomTable** RENAME COLUMN AncienNom TO NouveauNom;	
Renommer une colonne (Version 02)	**ALTERTABLE NomTable** CHANGE COLUMN AncienNom TO NouveauNom;	ALTER TABLE Client CHANGE COLUMN Nom NomClient;

5. Définitions des contraintes :

La commande SQL **ALTER TABLE** permet de modifier la structure d'une table en ajoutant ou supprimant des contraintes, ou en activant/désactivant des contraintes existantes.

Opération	Syntaxe	Exemple
Ajouter une contrainte	ALTER TABLE NomTable ADD CONSTRAINT NomContrainte TypeDeContrainte(NomColonne);	**ALTER TABLE** Commandes **ADD CONSTRAINT** fk_client FOREIGN KEY (ClientID) REFERENCES Clients(ClientID);
Supprimer une contrainte	ALTER TABLE NomTable DROP CONSTRAINT *NomContrainte;*	**ALTER TABLE** Commandes **DROP CONSTRAINT** fk_client;
Activer une contrainte	ALTER TABLE NomTable ENABLE CONSTRAINT *NomContrainte;*	**ALTER TABLE** Commandes **ENABLE CONSTRAINT** fk_client;
Désactiver une Contrainte	ALTER TABLE NomTable DISABLE CONSTRAINT *NomContrainte;*	**ALTER TABLE** Commandes **DISABLE CONSTRAINT** fk_client;

5. Définitions des contraintes :

Types de contraintes courantes :

1. **`PRIMARY KEY`** : Un identifiant unique pour chaque ligne de la table.
2. **`FOREIGN KEY`** : Crée un lien entre deux tables.
3. **`CHECK`** : Impose une règle sur les valeurs qu'une colonne peut accepter.